


[SSA Home](#)
[Data Overview](#)
[Schema
browser](#)
[Data access](#)
[Radial](#)
[Menu query](#)
[Freeform SQL](#)
[CrossID](#)
[Cookbook](#)
[Field 287](#)
[Personal SSA](#)
[Q&A](#)
[Release
History](#)
[Downtime](#)
[Links](#)
[Credits](#)


IFA ROE

[Home](#) | [Overview](#) | [Browser](#) | [Access](#) | [Cookbook](#) | [Links](#) | [Credits](#)


An Introductory SQL Tutorial for SSA Users

Contents

- [1. Introduction](#)
- [2. Primer](#)
 - [2.1 Relational databases](#)
 - [2.2 The SSA as a relational database](#)
 - [2.3 Structured Query Language \(SQL\)](#)
 - [2.4 The basic structure of an SSA SQL statement](#)
 - [2.5 Null and default values in the SSA](#)
 - [2.6 The *SELECT* statements defining the SSA views](#)
- [3. Reference: additional options in *SELECT* statements](#)
 - [3.1 Aggregate functions](#)
 - [3.2 Spatial functions](#)
 - [3.3 Mathematical functions](#)
 - [3.4 Operators](#)
- [4. Examples: 20 queries used in the development of the SSA](#)
- [5. The Field 287 database](#)
 - [5.1 Relational organisation](#)
 - [5.2 Source environment from the neighbour table](#)
 - [5.3 Cross-neighbours](#)
 - [5.4 Selection of variability candidates](#)
 - [5.5 Light curves](#)
- [6. The 2MASS Photometric Redshift Catalogue database](#)
 - [6.1 Matching 2MPZ to other data](#)

1. Introduction

In this document we provide a brief introduction to the use of Structured Query Language (SQL) for accessing data in the SuperCOSMOS Science Archive (SSA). We divide this document into a [Primer](#) aimed at users new both to the SSA and to SQL, a [Reference](#) which should be of use more experienced and returning users, and an [Examples](#) section, which presents a set of 20 realistic queries used in the design of the SSA. Readers wanting a fuller introduction to SQL should consult [an online tutorial](#) or one of the [legion of SQL books](#) available: [O'Reilly's SQL in a nutshell](#) is a good introduction. Some familiarity with SuperCOSMOS and the set of parameters returned by its image analyser are assumed in what follows, so readers may wish to consult the introduction provided by [Hambly et al. \(2001\), MNRAS, 326, 1295](#), while those wishing to know the differences between the data presented in the SSA and those made available previously through the [SuperCOSMOS Sky Survey web interface](#) should consult the [SSA Database Overview](#) page.

The SSA is a large database - more than 2 TB in size - so, for test purposes, we have produced the "Personal SSA" (PSSA), which is a small subset of the SSA, containing solely the data in the region of the sky with $184 < \text{RA (deg)} < 186$ and $-1.25 < \text{Dec. (deg)} < 1.25$, which is the same area of sky as the ["Personal SkyServer"](#) produced for the Early Data Release ([EDR](#)) of the Sloan Digital Sky Survey ([SDSS](#)). The PSSA may be downloaded from [here](#) (as a .zip file with installation instructions included) or can be queried using a [web interface](#) of the same form as for the full SSA. **SSA users are strongly encouraged to use the PSSA for developing and debugging queries that they want to run on the SSA:** with a database as large as the full SSA it can take a long time to find out that the query you wrote does not do what you intended!

In particular, queries within this tutorial may be run on the PSSA by simply copying the highlighted text from this document and pasting it into the text box of the PSSA's [SQL Query form](#), and after each one displayed against a highlighted background we provide a link to a copy of the output page obtained from running the query with the form's default setting of returning to the user's browser the first 30 rows of the result set. Note that these output pages may differ in detail from the ones you may obtain running the same query. As discussed below, SQL is a set-based language, and all that is guaranteed is that the same query on the same database returns the same result set, with no guarantees as to the order in which rows appear in that result set. The amount of data that users can have returned from the SSA is currently limited to 15 million row-columns - i.e. for a query returning five columns, a maximum of three million rows will be returned. The *COUNT(*)* command, described in Section 3.1.2 below, can be used to determine the number of rows to be returned by a query, if there is doubt whether this limited will be exceeded.

2. Primer

2.1. Relational databases

The SSA is a relational database, which means that it stores data in *tables* composed of *rows* and *columns*. Each row comprises the information stored for one data entry – i.e. a celestial object in the case of the SSA – and there is one column for each of the *attributes* recorded for that entry – e.g. RA, Dec, ellipticity, etc, for the SSA. The different tables comprising a database may be linked (or *related*), if they each have columns representing the same data value, and *integrity constraints* can be included in the table definitions which ensure consistency between two related tables, e.g. by preventing the deletion of only one of a pair of rows in different tables thus linked. For ease of use, it is possible to define virtual tables - called *views* - which are subsets of the data in one or more tables and which can be queried using the same syntax as ordinary tables (which are sometimes called *base tables*, to distinguish them from these virtual tables). In addition to tables and views, the major constituents of a relational database are *indexes* (the database community prefer that spelling to "indices"), which can speed up the identification of records which satisfy the particular condition expressed in a query, and various *stored procedures* and *functions* which extend the range of operations which can be performed on data held in the tables. The collection of definitions of columns, tables, views, indexes, stored procedures and functions in a database is called its *schema*.

2.2. The SSA as a relational database

The SSA schema is described in detail [elsewhere](#), but we recap here the basic features which we shall use later. The two major tables in the SSA are called *Detection* and *Source*. The columns in *Detection* are basically the attributes derived by running the SuperCOSMOS image analyser over a single plate scan, and these single-plate detections are then merged into multi-epoch, multi-colour records for individual celestial objects, which are stored in *Source*. In addition to these two major tables, there are also a number of metadata tables, which store ancillary information describing the processes involved in obtaining and reducing SuperCOSMOS data, and which enable the provenance of data values in *Source* and *Detection* to be traced all the way back to a given glass plate exposed in an observation of a particular survey field made under known conditions and subsequently processed using a certain set of calibration coefficients. The SSA uses the same set of spatial access routines as the [SDSS SkyServer](#), based on the Hierarchical Triangular Mesh ([HTM](#)) pixelisation of the celestial sphere, which was developed at Johns Hopkins University. To aid spatial matching of objects within the SSA and between the SSA and the SDSS EDR, respectively, there are also "Neighbours" and "CrossNeighboursEDR" tables which record pairs of sources within 10 arcsec of one another. Three views are defined in v1.0 of the SSA: *ReliableStars*, *CompleteStars* and *ReliableGalaxies*. As their names suggest, these are intended for use when well

defined subsamples of stars or galaxies with high completeness or reliability are required, and they are defined in terms of selections on attributes in the *Source* table. Their advantage is that the user does not need to remember the constraints (detailed in [Section 2.6](#) below) on the attributes required to define the subsample, but can simply query it using the view created to constitute that subsample. Users should check which attributes in which tables have been indexed in the v1.0 SSA, since the performance of queries that can make use of them should be significantly better than for those which do not: this information is presented in the [Schema Browser](#).

2.3. Structured Query Language (SQL)

SQL is the standard language for accessing and manipulating data stored in a relational database. In fact, several versions of the SQL standard exist, and most database management systems (DBMSs) actually support a subset of standard SQL, with some vendor-specific additions. The SSA is currently implemented in Microsoft's [SQL Server 2008](#) DBMS, so SSA users will employ its SQL dialect, although we have tried to restrict the use of vendor-specific features to a minimum. A fuller reference on this SQL dialect than presented here is available on line [here](#).

The first thing to understand about SQL is that it is a set-based language, not a procedural language, like Fortran or C. A user submitting an SQL query to a relational database is defining the set of properties of the records that she wants returned from the database, not specifying the list of operations which will lead to their delivery; this latter is the responsibility of the DBMS engine, which will decide the best way to execute a given query from a set of possible execution plans. Many database vendors are adding procedural capabilities to the SQL dialects they support, and these constitute one of the main areas of difference between those dialects. These extensions will not be discussed here, as we shall concentrate on the basics of standard SQL.

2.4. The basic structure of an SSA SQL statement

For security reasons, the SSA does not allow users to execute queries which affect the basic structure and contents of the database, only those which can extract data from it. In SQL terms, this means that only *SELECT* statements are allowed: *N.B.* in this tutorial we write all SQL keywords in upper case italics and some column names in mixed case, both for clarity, although the SSA's SQL dialect is case insensitive by default. There are three basic classes of *SELECT* statement:

2.4.1 Projections

A projection is the retrieval of a set of full columns from a table. To retrieve the nominal RAs and Decs of the centres of *all* sky survey fields in the SSA, one would type:

```
SELECT nominalRA, nominalDec FROM Field
```

[\[Link to demo result set\]](#)

where *Field* is the name of the SSA table which records information about sky survey fields, and *nominalRA* and *nominalDec* are the names of the relevant columns in that table.

2.4.2 Selections

A selection is the retrieval of the data values in particular columns for those rows in a table which satisfy certain criteria. So, if one were interested only in fields whose nominal centres lie in a 10 degree strip south of the celestial equator, the appropriate SQL query would be:

```
SELECT nominalRA, nominalDec
```

FROM Field

WHERE nominalDec *BETWEEN* -10 *AND* 0

[\[Link to demo result set\]](#)

In this example the SQL statement has been split into three lines to emphasise the *SELECT...FROM...WHERE* form of the selection, but this is still one SQL statement. The **SQL Query Form** in the SSA interface ignores the whitespace at the end of each line of text and generates a single query string from valid multi-line text like this. (Note that this means that users should *not* use double hyphens to indicate comments in multi-line queries, since this will result in all text after the first comment being ignored.)

Multiple constraints can be included in the *WHERE* clause of a selection, so, for example, the query above could be rewritten as:

```
SELECT nominalRA, nominalDec FROM Field WHERE (nominalDec > -10) AND
(nominalDec < 0)
```

[\[Link to demo result set\]](#)

while the field centres of all other fields could be selected using the following statement:

```
SELECT nominalRA, nominalDec FROM Field WHERE (nominalDec < -10) OR
(nominalDec > 0)
```

[\[Link to demo result set\]](#)

The parentheses in these examples have been included for clarity – they are only *required* when needed to avoid ambiguity, and when necessary to over-rule the standard order of precedence amongst operators, outlined in **Section 3.4.9**. (Users should note that the accidental omission of the *WHERE* clause from a selection turns it not into an invalid query, but into the projection of the columns contained in its *SELECT* clause, which, for tables as large as the *Source* and *Detection* tables of the SSA - both of which have in excess of one billion rows - will return a lot of data.)

2.4.3 Joins

A join is the retrieval of data entries from one or more tables in a database matched under some criterion. Extending our example above, a user may be interested in the dates on which SSA exposures in this equatorial strip were taken. The *Plate* table in the SSA has an attribute called *MJD*, which records the Modified Julian Date at the midpoint of the exposure of each photographic plate making up the SSA. The *Plate* and *Field* tables are linked by having the common attribute *fieldID*, which is a unique identifier for each sky survey field (e.g. Field 1 in the ESO/SRC field system has a different *fieldID* value to Field 1 in the Palomar system). The SQL query retrieving the desired dates here would be:

```
SELECT mjd, nominalRA, nominalDec
FROM field, plate
WHERE (nominalDec BETWEEN -10 AND 0)
AND (field.fieldID = plate.fieldID)
```

[\[Link to demo result set\]](#)

In this query, records in the *Plate* table and the *Field* table are being joined on condition that their values for the *fieldID* attribute are equal. Since the *fieldID* attribute is present in both tables, it is necessary to distinguish between the *fieldID* columns in the two tables in the join condition, and this is done in the query above by prefixing the attribute name by the table name.

Another way of doing this is to use a prefix which is an alias for the table name, assigned using the *AS* keyword in the *FROM* clause of the join statement. In this way, the previous query can be rewritten as:

```
SELECT mjd, nominalRA, nominalDec
FROM field AS f, plate AS p
WHERE (nominalDec BETWEEN -10 AND 0)
AND (f.fieldID = p.fieldID)
```

[\[Link to demo result set\]](#)

Prefixes are not needed for the other attributes in this query (*mjd*, *nominalRA* and *nominalDec*) because they each occur in only one of the tables listed in the *FROM* clause, so they are unambiguous. The *AS* keyword can also be used to rename the attributes in the *SELECT* clause so that their names in the output result set differs from those by which they are known in the database table. For example, a user thinking that the column names referred to in the query above are a little cryptic could rewrite it as follows:

```
SELECT mjd AS MeanJulianDate, nominalRA AS PlateCentreRA, nominalDec AS
PlateCentreDec
FROM field, plate
WHERE (nominalDec BETWEEN -10 AND 0)
AND (field.fieldID = plate.fieldID)
```

[\[Link to demo result set\]](#)

and the columns returned in the output result set will be headed *MeanJulianDate*, *PlateCentreRA*, *PlateCentreDec*. This useful facility can be misused by the unwary or the very stupid. For example, it would be possible to have a query which started "*SELECT ra AS dec, dec AS ra*", which could confuse the subsequent analysis of the extracted result set. (In the particular case of extracting data in [VOTable](#) format from the SSA, the assignment of Unified Content Descriptors ([UCDs](#)) to columns - i.e. setting the values of the *ucd* attributes to *<FIELD>* elements in the metadata section at the start of the VOTable document - is performed on the basis of the column name in the output result set, so a user swapping *ra* and *dec*, as in the example above, would end up with the wrong UCD for both those two columns, causing problems were that VOTable to be used in conjunction with other Virtual Observatory tools reliant on UCDs. For the moment, users wishing to have UCDs included in VOTable output from the SSA must not rename table columns, while, more generally, it is left to the user to ensure that any renaming they do perform is sensible.)

It is also possible to write join queries in a way that makes it more explicit that they are joining the two tables, i.e. the example above becomes:

```
SELECT mjd, nominalRA, nominalDec
FROM plate AS p
JOIN field AS f ON (f.fieldID = p.fieldID)
WHERE (nominalDec BETWEEN -10 AND 0)
```

[\[Link to demo result set\]](#)

This is an *inner join*, meaning it only returns the (*mjd*,*nominalRA*,*nominalDec*) triplets for matched rows, but there are other types of join, which return different combinations of data. Firstly, there are *left* and *right outer joins* and *full joins*, all of which also return data from unmatched rows. Replacing the join clause above with

```
LEFT JOIN field AS f ON (f.fieldID = p.fieldID)
```

would return the *nominalRA* and *nominalDec* values for any unmatched rows in *Field* (the table on the *left* of the join clause) in addition to the (*mjd*,*nominalRA*,*nominalDec*) triplets for matched rows. Similarly, the *mjd* values of any unmatched rows of *Plate* (the table on the *right* of the join) would be returned in addition to the (*mjd*,*nominalRA*,*nominalDec*) triplets for matched rows by the use of the following join clause:

```
RIGHT JOIN field AS f ON (f.fieldID = p.fieldID)
```

Finally, a *full join*

```
FULL JOIN field AS f ON (f.fieldID = p.fieldID)
```

would return the appropriate attributes for the unmatched rows in both tables, as well as the matched rows.

2.4.4 Subqueries

The SQL Server dialect of SQL allows the construction of nested *SELECT*

statements, in which the *WHERE* clause includes a *subquery* which is itself a valid *SELECT* statement. For example, the join on *Field* and *Plate* in the previous Section could be rewritten in the following way:

```
SELECT mjd
FROM plate
WHERE fieldID IN
(SELECT fieldID FROM field WHERE (nominalDec BETWEEN -10 AND 0))
```

[\[Link to demo result set\]](#)

Note that the prefixes have been removed here, since the two occurrences of *fieldID* are now separate and no longer ambiguous. The second thing to notice is that the *nominalRA* and *nominalDec* attributes from the *FIELD* table have now been removed from the *SELECT* clause of the main query. This is because the *FIELD* table is only present in the subquery, and its attributes are no longer accessible to the main query. The one exception to this is its *fieldID* attribute, but that is only accessible by value - i.e. the subquery generates a list of *fieldID* values and matches between this and the *fieldID* column of the *Plate* table are made by use of the *IN* operator.

This query could also be written using a second logical operator, *ANY*, as follows:

```
SELECT mjd
FROM plate
WHERE fieldID = ANY
(SELECT fieldID FROM field WHERE (nominalDec BETWEEN -10 AND 0))
```

[\[Link to demo result set\]](#)

where the *ANY* operator is used to match rows in the *Plate* table with *any* of the rows in the output result set from the subquery which have the same *fieldID* value. Note that in both these subqueries formulations the list of attributes in the *SELECT* clause of the subquery must be consistent with the rest of the *WHERE* clause in the main query, since they are to be matched by value.

These last two queries illustrate the equivalence of *IN* and *= ANY*, but care must be taken if the logical operator *NOT* is added to the outer query, so that it seeks matches with the complement of the set for which matches are sought in the original query. The operator which is equivalent to *NOT IN* is not *< > ANY*, as one might initially expect, but rather *< > ALL* - where *ALL* is another logical operator, which evaluates to *TRUE* if all of a set of comparisons are *TRUE* - which is clear when one thinks through the logic of that query, but perhaps not immediately apparent.

The most common use for subqueries is to express complicated joins in a simpler fashion. Up to 32 levels of nesting are allowed, in principle, although memory restrictions may prevent that from being achieved in practice. To evaluate a complicated join as a nested series of queries would often be much slower, but the database engine should recognise the equivalence of the two forms of the query and recast it in the most efficient form as part of its query optimisation process. So, there should be no difference in the performance of queries submitted as complicated joins or as nested subqueries, and the latter are to be preferred if they help ensure that the query that is executed really is the one that the user wanted to run.

2.5. Null and default values in the SSA

Our discussion to this point has implicitly assumed that the values of the attributes corresponding to each column in every row in a database table are known. This need not always be the case, as a simple example from the SSA illustrates. The *Source* table in the SSA merges information about detections made on B, R and I band plates. A very blue object may well not be detected in an I band plate, as these are shallower than the B and R plates, so what should be written for such an object in the column of the *Source* table which records, say, the ellipticity of the I band detection? One answer would be a *null* value, which is a special type of entry to be included in a table if the value of a given attribute is not known (or is indeterminate or is not applicable) for a particular row. In designing v1.0 of the SSA we have decided

not to use nulls in these cases, but to define **default** values for use in these situations instead: e.g. in the example above, we would set the I band ellipticity in the *Source* table of a celestial object undetected in that band to be -9.999e9. Nulls and default values are semantically different: the query processor in a database management system (DBMS) recognises that a value marked as null is unknown or indeterminate and will not include it in, say, the computation of the mean value of all entries in a column, while, to the query processor, a default value is like any other value, and will include it, unless the user explicitly excludes it - e.g. by computing the mean magnitude only for those objects with magnitudes brighter than -99.0, in this case.

As a result of defining default values for some columns, there have to be dummy rows in some tables (i.e. rows for which every attribute takes the appropriate default value). The reason for this is illustrated by consideration of the *Source* and *Detection* tables in the SSA, whose design is detailed [here](#). The *Source* table includes attributes (called *ObjIDB*, *ObjIDR1*, *ObjIDR2*, *ObjIDI*) which store the values of the *ObjID* attribute in the *Detection* table of the records, respectively, of the B band, first epoch R band, second epoch R band and I band detections which have been merged into the *Source* record. In our example above, there is no I band detection merged into the *Source* record of our very blue object, so *ObjIDI* must take a default value. This, in turn, necessitates the creation of a dummy row in the *Detection* table, whose *ObjID* attribute takes the value used as the default for *ObjIDB*, *ObjIDR1*, *ObjIDR2* or *ObjIDI* used for a non-detection in a particular band, since otherwise the integrity constraints that ensure that every detection referenced in the *Source* table is present in the *Detection* table would be violated. The other columns of this dummy row are then populated with the appropriate default values. The default values used for the attributes in the SSA are listed [here](#) and have all been chosen to lie well beyond the range of legitimate values found in the SSA, so it is simple to exclude them.

It is the responsibility of the user to ensure that the presence of default values and dummy rows will not corrupt the results of queries run on the SSA, but our decision to use them, rather than nulls greatly simplifies the logic involved in **all** queries run on the database. The inclusion of null values for an attribute means that an expression involving it can evaluate to *TRUE*, *FALSE* or *UNKNOWN*, and we believe that the simplification for users in avoiding this three-valued logic greatly outweighs the burden of having to remember that default values exist for some columns in the SSA.

2.6. The *SELECT* statements defining the SSA views

Having discussed the syntax of a *SELECT* statement, we can describe the definition of the views available in v1.0 of the SSA. As outlined above, views are virtual tables comprising a subset of data drawn from one or more tables. Views are defined using a *SELECT* statement and in the SQL Server dialect of SQL this takes the following form:

```
CREATE VIEW ViewName AS select_statement.
```

In essence, **ViewName** is being defined as the result set from a virtual query, specified by **select_statement**, which can take any of the forms outlined above.

The benefits of defining views can be illustrated by the example of the *CompleteStars* view in SSA v1.0, which is defined as follows:

```
CREATE VIEW CompleteStars AS
SELECT * FROM Source
WHERE chi2 < 5.0 AND Nplates > 1 AND
  ellipB < 0.3333333 AND qualB < 2048 AND ((prfstatB > -3.0 AND prfstatB < +4.5)
  OR prfstatB < -0.9e9) AND
  ellipR1 < 0.3333333 AND qualR1 < 2048 AND ((prfstatR1 > -3.0 AND prfstatR1 <
```

```
+4.5) OR prfstatR1 < -0.9e9) AND
ellipR2 < 0.3333333 AND qualR2 < 2048 AND ((prfstatR2 > -3.0 AND prfstatR2 <
+4.5) OR prfstatR2 < -0.9e9) AND
ellipl < 0.3333333 AND qual1 < 2048 AND ((prfstatl > -3.0 AND prfstatl < +4.5) OR
prfstatl < -0.9e9)
```

The *CompleteStars* view defines a highly complete selection of point sources from the *Source* table. It contains all the attributes for all those entries in *Source* for which there is a reasonable astrometric solution ($\chi^2 < 5.0$) derived from more at least two plates ($N_{\text{plates}} > 1$) and which, in all the bands in which it is detected, is free of strong warnings concerning quality issues ($\text{qual} < 2048$) and has image parameters within ranges outwith which it is highly unlikely that point sources will be found; these ranges are defined to be ellipticity $< 1/3$ (i.e. image elongation is less than 50%) and $-3.0 < \text{profile statistic} < 4.5$ (i.e. the areal profile set for the image is close to that of an idealised PSF, where "close" is between a 3sigma lower limit and a 4.5 sigma upper limit on the $N(0,1)$ profile statistic). Use of this view is recommended for analyses requiring high stellar completeness but which are not too sensitive to some contamination, particularly from misclassified sources, poorly parameterised deblends near the limiting magnitude of the plate and poorly parameterised/calibrated images in very crowded regions.

As always, the user must decide what SQL statement defines the set of sources best suited to the particular scientific problem at hand, and in some cases, reliable star/galaxy separation will be more important than completeness, and, with this in mind, we have defined *ReliableStars* and *ReliableGalaxies* views for SSA v1.0, using the following SQL statements:

```
CREATE VIEW ReliableStars AS SELECT * FROM Source
WHERE chi2 < 3.0 AND Nplates > 2 AND
ellipB < 0.25 AND blendB < 1 AND qualB < 128 AND ((prfstatB > -3.0 AND prfstatB
< +3.0) OR prfstatB < -0.9e9) AND
ellipR1 < 0.25 AND blendR1 < 1 AND qualR1 < 128 AND ((prfstatR1 > -3.0 AND
prfstatR1 < +3.0) OR prfstatR1 < -0.9e9) AND
ellipR2 < 0.25 AND blendR2 < 1 AND qualR2 < 128 AND ((prfstatR2 > -3.0 AND
prfstatR2 < +3.0) OR prfstatR2 < -0.9e9) AND
ellipl < 0.25 AND blendl < 1 AND qual1 < 128 AND ((prfstatl > -3.0 AND prfstatl <
+3.0) OR prfstatl < -0.9e9) AND
(b > +10.0 OR b < -10.0) AND d > 20.0
```

and

```
CREATE VIEW ReliableGalaxies AS SELECT * FROM Source
WHERE objidB > 0 AND objidR2 > 0 AND objidl > 0 AND
classB = 1 AND classR2 = 1 AND blendB = 0 AND blendR2 = 0 AND
qualB < 128 AND qualR2 < 128 AND qual1 < 128 AND gcormagB < 20.0 AND
d > 45.0 AND (b > +35.0 OR b < -35.0)
```

both of which combine cuts on image parameters with constraints on source position in Galactic coordinates, to exclude regions where star/galaxy separation is difficult: the *l*, *b* and *d* columns in the *Source* table store, respectively, the Galactic longitude and latitude of the source, and its distance (in degrees) from the Galactic centre.

The definition of the *ReliableStars* view is best understood by contrasting it with that for *CompleteStars*. The definition of *ReliableStars* imposes tighter constraints on many parameters than *CompleteStars* to reduce contamination from non-stellar sources: the maximum ellipticity is reduced from one third to one quarter; the upper limit on the profile statistic is reduced to 3.0sigma; the overall image quality must be better ($\text{qual} < 128$) and only non-deblended ($\text{blend}=0$) detections are considered. Sources must be detected on at least three out of possible four plates ($N_{\text{plates}} > 2$) and these multi-epoch detections must admit a good astrometric solution ($\chi^2 < 3.0$). Finally, a cut in Galactic coordinates is made. Sources must lie at $|b| > 10$ degrees, and be more than 20 degrees from the Galactic centre, to avoid the most crowded areas of the plane and bulge, where image parameterisation is difficult.

Note that the sample defined by this view will be incomplete in the most crowded regions included by this positional cut, due to blended stars being mis-classified as galaxies, so, if completeness is more of a concern than reliability, users may choose not to use *ReliableStars*, however it is recommended for those analyses where contamination by non-stellar sources must be minimised.

The selection defining the *ReliableGalaxies* view is simpler. It is a magnitude-limited ($B_j < 20.0$) subsample of sources with good quality ($qual < 128$) detections on UKJ, UKR and UKI plates, which are classed as non-stellar and non-blended in the UKJ and UKR surveys (which are deeper than the UKI) and which satisfy a positional constraint ($|b| > 35.0$ and $d > 45.0$) designed to excise those regions on the sky where contamination by stars in the bulge and close to the Galactic plane is a significant problem. As with the other views in the v1.0 SSA, use of *ReliableGalaxies* can greatly simplify the SQL for a great many of the queries to be run against the SSA, but users must be aware of its limitations, notably its incompleteness at faint magnitudes and lower Galactic latitudes.

3. Reference: additional options in *SELECT* statements

Previous sections have described the basic *SELECT...FROM...WHERE...* structure of an SQL query. This is the basic syntax to be employed for querying the SSA, but there are some additional options in the *SELECT* clause which users may find useful.

3.1 Aggregate Functions

SQL offers a number of useful aggregate functions, which can be used for deriving summary information on columns or selections from them.

3.1.1 Mathematical aggregate functions

The meanings of those mathematical aggregate functions which apply only to numeric columns are very straightforward: *AVG*, *MAX*, *MIN*, *SUM*, *STDEV*, *STDEVP*, *VAR*, and *VARP* return, respectively, the mean, maximum, minimum, sum, standard deviation, population standard variation, variance and population variance of all values in the specified column. They can be used in conjunction with a *WHERE* clause or not, i.e.

```
SELECT max(nominalDec) FROM field
```

[\[Link to demo result set\]](#)

will return the maximum value for the nominalDec column found in the field table, while

```
SELECT max(nominalDec) FROM field WHERE (nominalRA BETWEEN 0 AND 180)
```

[\[Link to demo result set\]](#)

returns the maximum values found for fields within one hemisphere of the sky.

N.B. Strictly speaking, these functions only apply to *non-null* values found within the particular column. As discussed above, the v1.0 SSA contains no null values by design, but uses default values instead. These will not be ignored in the computation of aggregate functions. For example, the SQL query

```
SELECT min(nominalDec) FROM field
```

[\[Link to demo result set\]](#)

returns the value -999.9999990, which is clearly nonsensical astronomically, and just illustrates the unintentional selection of the dummy row in the *Field* table.

3.1.2 COUNT

There are additional aggregate functions which can be run on columns of all types. The most useful of these is *COUNT*, which can be used in a number of ways, as shown in the following example. The *systemID* attribute in the *Field* table identifies the scheme under which the field centres were defined: since the SSA combines data from several surveys, it can take several different values. To find out how many, one can use the following query:

```
SELECT COUNT (DISTINCT systemID) FROM field
```

[\[Link to demo result set\]](#)

which returns the answer 4 (which includes the default value of zero, and which could, in practice, also be discovered from querying the SSA's *FieldSystem* table). Note that the inclusion of the *distinct* keyword means that only the number of distinct *systemID* values was returned. If it had been omitted, then the returned value would be the total number of entries in the *systemID* column, which is simply equal to the number of rows in the table, since there are no null values for *systemID* in the *Field* table.

The total number of rows in the table could be obtained using the following query:

```
SELECT COUNT(*) FROM field
```

[\[Link to demo result set\]](#)

The *count(*)* function can also be used in conjunction with a *WHERE* clause, and it is in this guise that it is most useful. Suppose a user were interested in selecting a sample of the very large objects from the *Detection* table, but didn't know the size of the largest objects. In ignorance, the user might think that 100 pixels was a pretty large area and would query the SSA to return a list of attributes for all such objects. Such a query would return about one tenth of the records in the *Detection* table, which would probably return far more data than the user could cope with. A better policy would be to use *count(*)* to find out how many objects would be returned by an area cut at 100 pixels, with the following query:

```
SELECT COUNT(*) FROM detection WHERE area > 100
```

[\[Link to demo result set\]](#)

and if the count returned by this query is too high, it can be repeated with a higher area cut until a sensible sample size is returned - e.g. a cut a 1000 pixels returns something like 0.5% of the entries in *Detection*. Using *COUNT(*)* like this is a very good way of ensuring that a query is sensible before getting data returned by running it, and users are strongly encouraged to use this method in conjunction with the PSSA to develop and debug SQL queries before running them on the full SSA. The performance overhead involved in this two-query process is not as high as it might seem, since, depending on how heavily the database is being used, some of the data from the *COUNT(*)* query will still be in cache when the query is run a second time to extract the desired attributes. Users should note that *COUNT* should be replaced by *COUNT_BIG* if there is likely to be more than 2×10^9 records to be counted: this circumvents a SQL Server bug.

3.1.3 TOP

An interesting function specific to SQL Server's SQL dialect is *TOP*, which is illustrated as follows. The query

```
SELECT TOP 10 area FROM detection
```

[\[Link to demo result set\]](#)

would return the area values for ten rows in the *Detection* table. This will generally *not* be the ten highest area values in the table; remember that SQL is a set-based language, so a query yields the set of rows satisfying the criteria specified in the query, but with, by default, no particular ordering within that set. The ten highest area

values can be obtained using *TOP*, however, with the addition an *ORDER BY* clause, which does impose an ordering of the rows in the result set: *i.e.*:

```
SELECT TOP 10 area FROM detection ORDER BY area DESC
```

[\[Link to demo result set\]](#)

Note the presence of the *DESC* (for descending) keyword in the *ORDER BY* clause, which is required because the default behaviour is for that clause to list rows in ascending order.

3.1.4 GROUP BY and HAVING

The *GROUP BY* clause allows aggregate functions to return more than a single value. For example, the user running the "*SELECT COUNT(DISTINCT systemID) FROM field*" query above might want to know how many of the SSA fields come from each of the four systems whose presence is indicated by the result of that query. That information is returned by the following query:

```
SELECT systemID, COUNT(*) FROM field GROUP BY systemID
```

[\[Link to demo result set\]](#)

where the rows in *field* are grouped by their *systemID* value and separate counts are made of the number of rows in each group. The result set from this query includes the dummy row in the *Field* table (which has *systemID*=0), and this may be removed through the use of the *HAVING* clause, *i.e.*

```
SELECT systemID, COUNT(*) FROM field GROUP BY systemID HAVING
systemID > 0
```

[\[Link to demo result set\]](#)

where the *HAVING* clause here adds a search condition on the results of the *GROUP BY* clause.

3.2 Spatial Functions

The majority of queries run on astronomical databases have a spatial component to them, restricting attention to particular region of the celestial sphere, usually defined as some range in either Equatorial or Galactic coordinates. To enhance the performance of the database for these popular spatial queries would require some sort of spatial indexing, but a practical problem is that very few DBMSs support two-dimensional indexing on the surface of a sphere. Indeed, not all DBMSs support two-dimensional indexing on a plane, and SQL Server is one of those DBMSs which supports indexing in only one dimension. Indexing on RA and/or Dec (which is better, not having the complication of wrap-around at the meridian) would aid spatial queries somewhat - and, indeed, indexing on both RA and Dec is implemented in the v1.0 SSA - but what is desired is a prescription under which two-dimensional regions of the celestial sphere can be indexed using a one-dimensional index such that regions which are close on the sky have similar values for the index code, as that will facilitate efficient spatial searches.

3.2.1 The HTM Scheme

A number of approaches can be used to map the two-dimensional surface of a sphere onto a one-dimensional index, but, as mentioned above, the v1.0 SSA uses the same spatial indexing prescription as the [SkyServer](#) database of the Sloan Digital Sky Survey ([SDSS](#)). This is based on the Hierarchical Triangular Mesh (HTM) pixelisation of the celestial sphere, which was developed at Johns Hopkins University by Peter Kunszt, Alex Szalay and Ani Thakar, and implemented in SQL Server by Jim Gray and Don Slutz of Microsoft Research: we thank them for making available their [code for use in the SSA](#). The HTM scheme is described in much greater detail [elsewhere](#), and we discuss here only those facets which are relevant to use of the SSA.

The HTM scheme starts with the projection of an octahedron onto the celestial

sphere, which defines a base set of eight triangular regions of the sky. Each triangular face of the octahedron is then divided into four triangles, defined such that their vertices are the vertices of the original triangle and the midpoints of its sides. The projection of this set of triangles onto the sphere divide it into 32 regions. This process of splitting triangles into four can be repeated to yield an increasingly finely-grained pixelisation of the sphere. The triangles defined at each stage in this procedure can be numbered according to a set procedure, in which an integer code is assigned to each whose most significant bits are inherited from the parent triangle and whose least significant bits record which of the four triangles created from that parent it is. In this way, it is possible to define an integer pixel code - the *HTMID* - for each triangle in this hierarchical pixelisation of the sphere which records not only the location of the centre of the triangle on the celestial sphere, but also the level in the hierarchical decomposition of the sphere at which it is defined. A 20-level decomposition produces triangles whose average area (note that the areas on the sphere bounded by projections of the edges of the hierarchically decomposed octahedron are *not* equal) is less than 0.1 square arcsecond, so these are perfectly adequate for labelling individual celestial objects within the SSA unambiguously.

This 20-level HTM decomposition is the basis for the spatial indexing of all positions (locations of celestial objects in the *Detection* and *Source* tables, and of field centres in the *Field* table) in the v1.0 SSA: *N.B. all equatorial positions within the v1.0 SSA are quoted in J2000 coordinates.* The *HTMID* for all these defined positions is recorded in the *HTMTable* table, which includes columns for an ID code (the *TableID*, itself defined in the *HTMTableID* table) for the parent table in which the position is defined, together with the ID number within that table of the row in which it is found. Following SkyServer we provide a set of functions for manipulating the entries in *HTMTable*: these are the key to performing efficient spatial queries in the SSA, so we strongly recommend that users learn how to use them.

3.2.2 fHTMLookupEq and fHTM_Cover

The first two functions are concerned with the mapping from regions of the celestial sphere to the *HTMID* code. The function *fHTMLookupEq* takes as arguments an (RA,Dec) pair (with both coordinates in degrees) and returns the 20-level *HTMID* code for that location. For example, the *HTMID* for the point (185.0,0.0) is found with the following query:

```
SELECT dbo.fHTMLookupEq(185.0,0.0)
```

[\[Link to demo result set\]](#)

The prefix *dbo.* must be added to the names of *all* functions called on the SSA: it stands for "database owner" and is part of the system by which SQL Server assigns access rights to different user account names.

A more powerful function is *fHTM_Cover*, which returns the *HTMID* values of the triangles (at a depth in the hierarchical decomposition specified by the user) which intersect a particular region of the celestial sphere (defined in J2000 coordinates). This region can be specified in a number of ways, the most useful being a *CIRCLE*, defined by the (RA,Dec) pair of its centre (both in degrees) and its radius (in arcminutes), and a *CONVEX*, which is a polygon defined by the (RA,Dec) pairs of its vertices. Whereas *fHTMLookupEq* returns a single integer (the *HTMID* code), *fHTM_Cover* is a *table-valued function*, meaning that it returns a table, which must be queried with a *SELECT* statement of the same syntax as the static tables and views which are found in the SSA schema. The use of *fHTM_Cover* is illustrated in the following query, which probes the coverage of the circular region of radius 40 arcminutes centred on (185.0, 0.0) by Level-6 HTM triangles:

```
SELECT * FROM dbo.fHTM_Cover("CIRCLE J2000 6 185.0 0.0 40")
```

[\[Link to demo result set\]](#)

This returns a table with six rows and two columns, the latter being headed with the names *HTMIDstart* and *HTMIDend*. The list of Level-6 HTM triangles which intersect

with the circular search region is obtained by adding together the six intervals which start and end, respectively, with the *HTMID* values listed in the *HTMIDstart* and *HTMIDend* columns: i.e. in this case, the list of triangles is [40968-40969,40996-40997,41012-41013,55300-55301,55320-55321,55352-55353]. Similarly, the use of the *CONVEX* form of *fHTM_Cover* is demonstrated by the following query:

```
SELECT * FROM dbo.fHTM_Cover("CONVEX J2000 8 184.6 0.3 184.4 0.3 185.2
-0.2 185.0 0.0")
```

[\[Link to demo result set\]](#)

which returns an eight-row table of (*HTMIDstart*,*HTMIDend*) pairs from which can be reconstructed the list of Level-8 HTM triangles which intersect the polygon with vertices at (184.6,0.3), (184.4,0.3), (185.2,-0.2) and (185.0,0.0).

3.2.3 fGetNearbyObjEq, fGetNearestObjEq and fGetNearestObjIDEq

The next three functions perform proximity searches within *HTMTable*. The first of these, *fGetNearbyObjEq* searches for entries from a particular parent table that lie within a circle of a given radius centred on a location specified by an (RA,Dec) pair. So, the following query:

```
SELECT * FROM dbo.fGetNearbyObjEq(185.0,0.0,0.5,1)
```

[\[Link to demo result set\]](#)

selects entries from the *Source* table (whose members have *tableID*=1 in *HTMTable*) within a radius of 0.5 arcmin of (185.0,0.0). *fGetNearbyObjEq* is a table-valued function, which returns a set of six attributes for each of the selected rows: the *ObjID* value for the row in its parent table (which is the attribute *fieldID* for plate centres from the *Field* table); the triplet (cx,cy,cz), which is a representation of the entry's position on the celestial sphere in the Cartesian coordinates used "under the hood" of this routine; the *HTMID* corresponding to that position; and the *distance* (in arcminutes) from that source position to the centre of the search region. Since *fGetNearbyObjEq* is a table-valued function, joins can be made with it, just as with any other table. So, one can find out whether any of the entries from *Source* returned by the previous query are contained within the *ReliableGalaxies* view with the following query:

```
SELECT * FROM dbo.fGetNearbyObjEq(185.0,0.0,0.5,1) AS n, ReliableGalaxies
AS g WHERE n.objID=g.objID
```

[\[Link to demo result set\]](#)

which checks for the presence in *ReliableGalaxies* of the entries in the result set of the *fGetNearbyObjEq* proximity search through a join on their *objID* value.

The ordering of the results returned by *fGetNearbyObjEq* is not guaranteed, so, to enable the ready determination of the closest entry in *Source*, say, to a given position on the sky without a second selection on the *distance* column of the result set, another function, *fGetNearestObjEq*, is defined to do just that. So, the nearest of the *Source* entries to position (185.0,0.0) is returned by:

```
SELECT * FROM dbo.fGetNearestObjEq(185.0,0.0,0.5,1)
```

[\[Link to demo result set\]](#)

where *fGetNearestObjEq* is another table-valued function, which returns the same set of six columns as *fGetNearbyObjEq*, while a related function, *fGetNearestObjIDEq*, returns only the ID value of the nearest entry. Since this is not table-valued, it is used with *SELECT* alone, rather than using a "*SELECT * FROM*" syntax - i.e the *ObjID* value of the entry in the *Source* table nearest to the position (185.0,0.0) is returned using the following query

```
SELECT dbo.fGetNearestObjIDEq(185.0,0.0,0.5,1)
```

[\[Link to demo result set\]](#)

3.2.4 fGreatCircleDist

One of the main purposes of introducing the HTM pixelisation and its associated set of access functions was to reduce the number of great circle distance calculations which have to be performed in proximity searches: the great circle distance formula is a fairly complex piece of spherical trigonometry, which is easy to type incorrectly in SQL and which is not efficiently executed by standard DBMSs, so it is desirable that its use be kept to a minimum. It is actually used in proximity searches, such as those specified using the *fGetNearbyObjEq* function, but only as a last check on the restricted number of rows which might match the proximity criterion on the basis of their *HTMID* value. However, the great circle distance formula is useful in many situation, so we define a function, *fGreatCircleDist*, that implements it. Its use is illustrated by the following query:

```
SELECT dbo.fGreatCircleDist(185.0,0.0,184.0,0.0)
```

[\[Link to demo result set\]](#)

which returns the great circle distance (in arcminutes) between (185.0,0.0) and (184.0,0.0).

3.3 Mathematical Functions

The following mathematical functions are supported by SQL Server's SQL dialect.

- **ABS(x)**: returns the absolute value of x .
- **ACOS(x)**: returns the angle, in radians and in the range 0 to π , whose cosine is x .
- **ASIN(x)**: returns the angle, in radians and in the range $-\pi/2$ to $\pi/2$, whose sine is x .
- **ATAN(x)**: returns the angle, in radians and in the range $-\pi/2$ to $\pi/2$, whose tangent is x .
- **ATN2(y,x)**: returns the angle, in radians, whose tangent is y/x .
- **CEILING(x)**: returns the smallest integer greater than, or equal to, x .
- **COS(x)**: returns the cosine of angle x in radians.
- **COT(x)**: returns the cotangent of angle x in radians.
- **DEGREES(x)**: returns the angle in degrees corresponding to angle x in radians.
- **EXP(x)**: returns the exponential of x .
- **FLOOR(x)**: returns the largest integer less than or equal to x .
- **LOG(x)**: returns the natural logarithm of x .
- **LOG10(x)**: returns the base 10 logarithm of x .
- **PI()**: returns the value of π .
- **POWER(x,y)**: returns the value of x raised to the power y .
- **RADIANS(x)**: returns the angle in radians corresponding to angle x in degrees.
- **RAND(x)**: returns a random number between 0 and 1, using an optional integer seed x if supplied.
- **ROUND(x,n,m)**: returns the value of x rounded to a specified length or precision. If n is a positive integer, then x is rounded to the number of decimal places specified by n . If n is a negative integer, then x is rounded n places to the left of the decimal point. *ROUND* can also be used to truncate x , through use of the optional argument m . If m is a positive integer, then x is truncated to the precision specified by n , rather than rounded to it.
- **SIGN(x)**: returns the value 1, 0, -1, depending whether x is positive, zero or negative, respectively.
- **SIN(x)**: returns the sine of the angle x in radians.
- **SQUARE(x)**: returns the square of x .
- **SQRT(x)**: returns the positive square root of x .
- **TAN(x)**: returns the tangent of angle x in radians.

Arithmetic functions (such as ABS, CEILING, DEGREES, FLOOR, POWER, RADIANS, and SIGN) return a value having the same data type as the input value, while trigonometric functions and others (such as EXP, LOG, LOG10, SQUARE, and SQRT), cast their input values to float and return a float value; this probably is of no concern to the average SSA user. All mathematical functions, except for RAND, are deterministic functions - i.e. they return the same results each time they are called

with a specific set of input values - and RAND is deterministic only when a seed parameter is specified

3.4 Operators

An operator in SQL is a symbol specifying an action that is performed on one or more expressions. For the present purposes, their major use is to provide greater flexibility in the possible forms of *WHERE* clauses of queries and in the columns of the result sets they can produce, which need not simply be columns drawn from the table(s) being queried. There are several classes of operator to consider.

3.4.1 Arithmetic operators

SQL Server's SQL dialect supports five arithmetic operators. The four basic ones - Addition (+), Subtraction (-), Multiplication (*) and Division (/) - plus the Modulo operation, (%), which returns the remainder of dividing one integer by another, and is used in the format "*dividend%divisor*". The use of the four basic arithmetic operators is demonstrated in [Section 4](#) below, and use of all five is straightforward from their definitions.

3.4.2 Bitwise operators

The utility of bitwise operators in SSA v1.0 is limited to the testing of quality flag in the *Detection* table, which is composed of flags for various quality issues set independently in a bitwise fashion, and which reappears as the set of four quality flags (qualB, qualR1, qualR2 and qualI) for the detections merged into each record in the *Source* table. The meaning of each of the quality flags set bitwise in these attributes is explained [here](#), and they may be tested using the Bitwise AND operator, &. For example, Q10 of [Section 4](#) probes the setting of the 4th bit of the quality flag (corresponding to presence of a pixel value above the highest areal profile level) by using terms in the *WHERE* clause of the form "qualb & 16 = 16". In addition, there are also Bitwise OR (|) and Bitwise Exclusive OR (^) operators defined, which could be used in a similar fashion.

3.4.3 Comparison operators

The Boolean comparison operators are used most frequently to filter rows via the *WHERE* clause of a SQL query. The most simple comparison operators (<, >, =) were used above without introduction, but there a total of nine comparison operators which can be applied to pairs of expressions in the SQL Server dialect of SQL: = (Equal to); > (Greater than); < (Less than); >= (Greater than or equal to); <= (Less than or equal to); <> (Not equal to); != (Not equal to); !> (Not greater than); and !< (Not less than).

3.4.4 Logical operators

In a similar fashion, we have used a number of the logical operators (e.g. *AND*, *BETWEEN*, etc) above without introduction, but the following is the full list of logical operators supported by SQL Server:

- **ALL**: TRUE if all of a set of comparisons are TRUE.
- **AND**: TRUE if both of two Boolean expressions are TRUE.
- **ANY**: TRUE if at least one of a set of comparisons is TRUE.
- **BETWEEN**: TRUE if the operand is within a specified range.
- **EXISTS**: TRUE if a subquery contains any rows.
- **IN**: TRUE if the operand is equal to one of a list of expressions.
- **LIKE**: TRUE if the operand matches a pattern.
- **NOT**: Reverses the value of any other Boolean expression.
- **OR**: TRUE if either of a pair of Boolean expressions is TRUE.

- **SOME**: TRUE if at least one of a set of comparisons is TRUE.

The use of some of these operators (e.g. *AND*, *BETWEEN*, *OR*) has been illustrated above, but it is worth making a few comments on the remaining ones.

The *LIKE* operator is used for pattern matching. This is most commonly used for string matching. For example, a user interested in knowing how many plates yielding data included in the SSA had the IIIaF photographic emulsion could issue the query:

```
SELECT COUNT(*) FROM plate WHERE emulsion LIKE "IIIaF"
```

[\[Link to demo result set\]](#)

Several wildcard characters can be used in conjunction with *LIKE*. For example, the grain size in the photographic emulsions changed with time, so someone might be interested in finding the plates with an emulsion from the IIIa family. The query for that would be:

```
SELECT COUNT(*) FROM plate WHERE emulsion LIKE "IIIa%"
```

[\[Link to demo result set\]](#)

where the percentage sign, %, is a wildcard for any string of zero or more characters. Three other wildcards can be used with *LIKE* in a similar fashion. The underscore, _, denotes a single character, so the previous query could also have been written

```
SELECT COUNT(*) FROM plate WHERE emulsion LIKE "IIIa_"
```

[\[Link to demo result set\]](#)

If the user wanted to be sure that only plates with the IIIaF or IIIaJ emulsions were selected, this could be amended to read

```
SELECT COUNT(*) FROM plate WHERE emulsion LIKE "IIIa[FJ]"
```

[\[Link to demo result set\]](#)

while plates with any other IIIa emulsion apart from those could be identified with this query:

```
SELECT COUNT(*) FROM plate WHERE emulsion LIKE "IIIa[^FJ]"
```

[\[Link to demo result set\]](#)

as the presence of the caret, ^, indicates that emulsion names ending in F or J should not be included in the count. A range of characters can be included by use of a hyphen, -, in the square brackets, so the query above could be rewritten as

```
SELECT COUNT(*) FROM plate WHERE emulsion LIKE "IIIa[^F-J]"
```

[\[Link to demo result set\]](#)

if one also wanted to exclude emulsions IIIaG, IIIaH, IIIaI, should they exist. This last question on its own can be determined using the following query:

```
SELECT "Filters IIIaG, IIIaH, IIIaI do not exist" WHERE NOT EXISTS (SELECT * FROM plate WHERE emulsion LIKE "IIIa[G-I]")
```

[\[Link to demo result set\]](#)

This query returns as a result set the string "Filters IIIaG, IIIaH, IIIaI do not exist" when run on the SSA, and is the first query we have seen that do not include a *FROM* clause; this is not required because the data to be returned, as specified in the *SELECT* clause, is a constant, rather than a set of values drawn from a table.

The *IN* logical operator determines if a given value matches any entries in a subquery or list. The first of these is discussed in Section 2.4.4. above [\[add link\]](#), while the second is readily illustrated by another reworking of the query above to count all plates taken with IIIaF or IIIaJ emulsions. Using *IN* instead of *LIKE* it can be rewritten as:

```
SELECT COUNT(*) FROM plate WHERE emulsion IN ("IIIaF", "IIIaJ")
```

[\[Link to demo result set\]](#)

The *SOME* and *ANY* operators are strictly equivalent and both compare a scalar expression with a single column set of values. The count of IIIaF plates could be found (in a rather circuitous manner) with the following query:

```
SELECT COUNT(*) FROM plate WHERE "IIIaF" = ANY (SELECT emulsion FROM plate)
```

[\[Link to demo result set\]](#)

Many of the examples given in this Section have been somewhat contrived, as we have struggled to find realistic astronomical queries that would make use of some of the logical operators supported by the SQL Server dialect of SQL. This probably indicates that the average SSA user is unlikely to make use of these operators, which we have discussed here solely for the sake of completeness.

3.4.7 String concatenation operator

The plus sign, +, is used as a string concatenation operator in the SQL Server dialect of SQL. This is most likely to be of use to SSA users in the formatting of result sets - i.e. in the definition of *SELECT* clauses.

3.4.8 Unary operators

The SQL Server dialect of SQL defines three unary operators - i.e. operators which have only one operand - although none of these are likely to be of much use to most SSA users. The first of these is the positive unary operator, +, which is used principally in SQL statements not allowed by the SSA query interface, so this will not be of use to SSA users. The second, -, the negative unary operator, which returns the negative value of a numeric expression, as shown in the following query:

```
SELECT -min(decmin) FROM survey
```

[\[Link to demo result set\]](#)

The final unary operator, ~, the Bitwise NOT operator, converts each bit in a numeric expression of integer data type into its 1s complement (i.e. 0s are changed to 1 and vice versa). This might be of use in queries involving the quality flags in the *Source* and *Detection* tables.

3.4.9 Operator precedence

The operators described in this subsection have the following descending levels of precedence:

- + (Positive Unary Operator), - (Negative Unitary Operator), ~ (Bitwise NOT)
- * (Multiply), / (Division), % (Modulo)
- + (Add), + (Concatenate), - (Subtract), & (Bitwise AND)
- =, >, <, >=, <=, <>, !=, !>, !< (Comparison operators)
- ^ (Bitwise Exclusive OR), | (Bitwise OR)
- NOT
- AND
- ALL, ANY, BETWEEN, IN, LIKE, OR, SOME

When two operators in an expression have the same operator precedence level, they are evaluated left to right based on their position in the expression.

4. Examples: 20 queries used in the development of the SSA

In their design of SkyServer, the SQL Server implementation of the SDSS archive, the Sloan archive team were influenced by a set of 20 queries (see [Data Mining the SDSS SkyServer Database](#) by Jim Gray et al.) which were intended to be realistic examples of the kinds of queries that astronomers would want to run on SkyServer. We adopted the same approach in our design of the SSA. We took the 20 queries used for SkyServer, translated a number of them to run on the SSA, and replaced

the others by new queries, often performing joins between tables in the SSA and others in the Early Data Release (EDR) version of SkyServer. We discuss these below in turn. Since these were intended to be representative of the queries which SSA users would submit, they have not necessarily been written in particularly elegant SQL, nor has the astronomical content (e.g. colour cuts used) been verified terribly rigorously. In each case we provide a link to the output obtained by running the query on the Personal SSA. To ensure that this matches exactly what a user would obtain we have amended some queries slightly, by the addition of a "TOP 30" to the select statement and an "...ORDER BY..." clause at the end of the query: recall from the discussion of *ORDER BY* above that the set-based nature of SQL means that the ordering of rows within a result set is not guaranteed, until explicitly dictated through use of *ORDER BY*.

Q1: Find the positions of all galaxies brighter than magnitude 20 in B with a local B band extinction is >0.1 mag.

The *Source* table of the SSA contains E(B-V) values computed from the [Schlegel et al. \(1998\), ApJ, 500, 525](#) maps for the position of each entry, so this query is simply a selection on the *EBmV* attribute recording those values scaled to yield the A_B . The scaling factor is 4.09 ([Binney and Merrifield 1998](#)) and we use the *ReliableGalaxies* view of the *Source* table, for simplicity, as that already makes our magnitude cut at $B=20$.

```
SELECT TOP 30 ra,dec
FROM ReliableGalaxies
WHERE ebmv*4.09 > 0.1
ORDER BY ra,dec
```

[\[Link to demo result set\]](#)

Q2: Provide the positions and magnitudes of stars for which the magnitudes from the two R band surveys differ by more 3 magnitudes.

This simple query performs a selection on *Source* to look for star-like objects detected in both red surveys in the SSA with magnitudes differing by more than 3 mags. Stars varying this much are rare, so this query is more likely to detect oddities in the data, rather than extreme variable stars. Indeed, run on the *CompleteStars* view in the PSSA it finds no matches at all, so, to see if it detects any artefacts in the PSSA we can relax the selection criteria for stars from those used to define the *CompleteStars* view, as follows:

```
SELECT TOP 30 ra,dec, sCorMagB, sCorMagR1, sCorMagR2, sCorMagI
FROM Source
WHERE
ellipR1 < 0.33 AND qualR1 < 2048 AND (prfstatR1 > -5 AND prfstatR1 < +5) AND
ellipR2 < 0.33 AND qualR2 < 2048 AND (prfstatR2 > -5 AND prfstatR2 < +5) AND
ABS (sCorMagR1-sCorMagR2) > 3 AND scorMagR1 > 0.0 AND sCorMagR2 > 0.0
ORDER BY ra,dec
```

[\[Link to demo result set\]](#) Even relaxing the cut on the profile statistic to +/-5 only yields one match within the PSSA area.

Q3: Find the positions of all galaxies with a profile statistic > 10 in all detected wavebands and photometric colours consistent with being an elliptical galaxy.

This query is a translation to the SSA of Query 5 from the SDSS list of "20 Queries", which selects sources for which a de Vaucouleurs profile is a better fit than an exponential profile. The profile information available in the SSA is more limited, so, in adapting this query to the SSA, we base it around a colour selection on the

ReliableGalaxies view together with a cut on the profile statistic which should only select large galaxies with profiles very dissimilar from the assumed PSF. For our colour cut, we take the original colour-magnitude locus for low-redshift ellipticals defined in (g,r,i) space by Eisenstein et al. (2001), AJ, 122, 2267 and translate it into cuts in SuperCOSMOS (B,R,I) space through the use of the transformations published by Fukugita et al. (1996), AJ, 111, 1748 between the Sloan and Johnson-Cousins passbands, and by Blair and Gilmore (1982), PASP, 94, 742 for conversion between them and the photographic filters used in the SSA, i.e.

- Eisenstein et al (2001) define the locus occupied by low-redshift ($z < 0.37$) Luminosity Red Galaxies in Sloan (g,r,i) colour-magnitude space as the following:
 $0.2 > (r-i) + 0.25*(g-r) - 0.18 > -0.2$ and $r < 13.1 + 2.33*(g-r) + 4 [(r-i)-0.18]$
- Fukugita et al. (1996) list the SDSS/Johnson-Cousins colour equations as
 $g-r = 1.05*(B-V) - 0.23$, $r-V = 0.49*(B-V) + 0.11 = V - 0.84*(V-R) + 0.13$ and
 $r-i = 0.98*(R-I) - 0.23$ for $R-I < 1.15$ and $r-i = 1.40*(R-I) - 0.72$ for $R-I \geq 1.15$
- Blair and Gilmore (1982) quote the following colour equation for photographic blue (BJ): $BJ = B - 0.28*(B-V)$, with negligible differences between the photographic and photometric R and I bands for sources without very red colours.

Putting all these together gives the following query, where we neglect the first epoch R band survey, since the definition of *ReliableGalaxies* ensures detection of a non-stellar source in the other three bands and relax the colour-magnitude cut, to allow selection of lower luminosity ellipticals than the LRGs of interest to Eisenstein et al. (2001):

```
SELECT TOP 30 ra, dec FROM ReliableGalaxies
WHERE (prfStatB>10 AND prfStatR2>10 AND prfStatI>10)
AND
(
(
(gCorMagR2-gCorMagI < 1.15) AND
(0.98*(gCorMagR2-gCorMagI)+0.20*(gCorMagB-gCorMagR2)-0.38 BETWEEN -0.2
AND 0.2) AND
(gCorMagR2<13.35+1.82*(gCorMagB-gCorMagR2)+3.92*(gCorMagR2-gCorMagI))
)
OR
(
(gCorMagR2 - gCorMagI >=1.15) AND
(1.40*(gCorMagR2-gCorMagI)+0.20*(gCorMagB-gCorMagR2)-0.87 BETWEEN -0.2
AND 0.2) AND
(gCorMagR2<11.39+1.82*(gCorMagB-gCorMagR2)+5.60*(gCorMagR2-gCorMagI))
)
) ORDER BY ra,dec
[Link to demo result set]
```

Q4: Provide the mean positions and magnitudes of any stellar objects with colours and proper motions consistent with being a white dwarf.

This query, which we shall run on the *ReliableStars* view selects white dwarf candidates on the basis of their Reduced Proper Motion, $Hr = R + 5 * \log_{10}(\mu/\text{arcsec per year}) + 5$, computed using proper motion (μ) from the SSA and R band photometry from the second epoch survey. White dwarfs are then selected as satisfying $Hr > 3.75*(B-R2) + 13.75$ and $Hr > 9.091*(R2-I) + 14.09$, and having significantly detected proper motions (i.e. $\mu > 5 \sigma_\mu$) with the following query:

```
SELECT TOP 30 ra,dec,sCorMagB,sCorMagR2,sCorMagI
FROM ReliableStars
```

WHERE

```
( ( SQUARE (muacosc) + SQUARE (mud)) > 5.0 * SQRT ( SQUARE
(muacosc*sigmuacosc) + SQUARE (mud*sigmud) ))
AND (sCorMagR2 + 5*LOG10(SQRT(SQUARE(muacosc/1000.0)+SQUARE(mud
/1000.0)))+5>3.75*(sCorMagB-sCorMagR2)+13.75)
AND (sCorMagR2 + 5*LOG10(SQRT(SQUARE(muacosc/1000.0)+SQUARE(mud
/1000.0)))+5>9.09*(sCorMagR2-sCorMagI)+ 14.09)
ORDER BY ra,dec
```

[\[Link to demo result set\]](#)

Q5: Find the positions and (B,R,I) magnitudes of all star-like objects within delta mag of 0.2 of the colours of a quasar of redshift $2.5 < z < 3.5$.

The corresponding query from the SDSS “20 Queries” list on which this is based sought quasar candidates in the redshift range $5.5 < z < 6.5$, which are very unlikely to be found in the SSA. All colour selection of quasars in the SSA will yield a large number of false positives, due to the lack of U band data, but we take the (g,r,i) selection criteria for low-z quasars from Richards et al (2001) – i.e. $-0.2 < (g-r) < 0.35$ and $-0.4 < (r-i) < 0.4$ – and translate that into the SSA photometric bands, using the tranformations from Q3, obtaining the following:

```
SELECT TOP 30 ra,dec, sCorMagB, sCorMagR2, sCorMagI
FROM ReliableStars
WHERE
(sCorMagB-sCorMagR2 BETWEEN 0.05 AND 0.80)
AND (sCorMagR2-sCorMagI BETWEEN -0.17 AND 0.64)
ORDER BY ra,dec
```

[\[Link to demo result set\]](#)

This selection criterion is not dissimilar to that of Irwin (1991) who argues that $z < 3$ quasars are to be found in an ellipse centred on $BJ-R \sim 0.4$, $R-I \sim 0.2$.

Q6 Find unpaired objects

This query implements one of the [Usages of the WFCAM Science Archive](#), itself devised from the [UKIDSS](#) proposal: the SSA is a prototype for the WFCAM Science Archive (WSA) in many respects. This query simply selects entries in *Source* which have a detection in the first or second epoch red survey, but not both.

```
SELECT TOP 30 objID, ra, dec
FROM Source
WHERE (objIDR1 > 0 AND qualR1<128 AND objIDR2=0) OR
(objIDR2 > 0 AND qualR2 < 128 AND objIDR1 = 0)
ORDER BY objID
```

[\[Link to demo result set\]](#)

Q7: Provide a list of star-like objects that are 1% rare in (B-R,R-I)-space.

This query illustrates how SQL can be used to generate rough density estimates in the space of the attributes present in a table. The query selects stellar objects on the basis of ellipticity and profile statistics and bins them up into a unit interval grid in (B-R,R-I) space.

```
SELECT TOP 30 CAST (ROUND((sCorMagB-sCorMagR2),0) AS INT ) AS RI,
CAST(ROUND((sCorMagR2-sCorMagI),0) AS INT) AS IZ, COUNT(*) AS pop
FROM ReliableStars
WHERE sCorMagB > 0 AND sCorMagR2 > 0 AND sCorMagI > 0
GROUP BY CAST(ROUND((sCorMagB-sCorMagR2),0) AS INT),
```

```
CAST(ROUND((sCorMagR2-sCorMagI),0) AS INT)
ORDER BY COUNT(*)
```

[\[Link to demo result set\]](#)

The *ROUND* functions round the colours into integer values, which are then turn into integer type by the *CAST(... AS INT)* operation. The output from this query is a table listing the number of entries from *ReliableStars* in each of the populated bins of (B-R,R-I) space, and these enable one to calculate which are the most unusual 1% of stars in that colour space.

Q8: Create a gridded count of galaxies with B-R>1.2 and R<19 over 184<RA<186 and -1.25<Dec<1.25, on a grid of 2 arcmin

This illustrates how a SQL query might be used in a clustering analysis, by computing counts-in-cells. As in Q7, it bins objects - this time into 2x2 arcmin bins on the sky – and then returns the count of galaxies in each bin.

```
SELECT TOP 30 CAST ((ra*COS(RADIANS(CAST (dec*30 AS INT)/30.0)))*30 AS
INT)/30.0 AS raCosDec,
CAST(dec*30 AS INT)/30.0 AS griddec, COUNT(*) AS pop
FROM ReliableGalaxies
WHERE (ra BETWEEN 184.0 AND 186.0) AND (dec BETWEEN -1.25 AND 1.25)
GROUP BY CAST ((ra*COS(RADIANS(CAST (dec*30 AS INT)/30.0)))*30 AS
INT)/30.0,
CAST(dec*30 AS INT)/30.0
ORDER BY racosdec,griddec
```

[\[Link to demo result set\]](#)

Q9: Create a count of galaxies in Level-9 HTM triangles which satisfy a certain colour cut, like 0.7B-0.5R-0.2I<0.8 and R<19.

This is another counting query, but this time, counting galaxies falling into particular Level-9 HTM triangles (regions about 1/4 square degree in size). This binning is performed by taking the *HTMID* for each relevant entry in the *ReliableGalaxies* view and dividing it by 2^{22} , as required to obtain the *HTMID* of the Level-9 triangle in which the galaxy lies, given that the *HTMID* stored in *ReliableGalaxies* is at Level-20.

```
SELECT TOP 30 (HTMID/POWER (2,22)) AS htm9, COUNT(*) AS pop
FROM ReliableGalaxies
WHERE
(0.7*gCorMagB-0.5*gCorMagR2-0.2*gCorMagI<0.8) and gCorMagR2<19
GROUP BY (HTMID/POWER (2,22))
```

[\[Link to demo result set\]](#)

Q10: Find the positions of all galaxies with a pixel brighter than the highest areal profile threshold in any band within 1 degree of a given point (185.0,0.0) in the sky

This query illustrates the use of a join with the result set from the table-valued function *fGetNearbyObjEq*. It is essentially a self-join of the *Source* table - i.e. a join of *Source* with itself - but uses the *ReliableGalaxies* view to restrict attention to galaxies. The call to *fGetNearbyObjEq* finds all objects in *Source* within 1 degree of (12 20 00, 00 00 00) - i.e. RA=185 degrees, Dec=0.0 degrees - and then joins with it a selection of those entries in *ReliableGalaxies* which are flagged in at least one band as having been detected with at least one pixel lying above the highest areal profile threshold used by the SuperCOSMOS Image Analyser. This flag is set with the 4th bit in the *qaflag*, and its value is probed using the Bitwise "AND" operator, &.

```
SELECT TOP 30 ra,dec
```

```
FROM ReliableGalaxies AS g JOIN fGetNearbyObjEq(185.0,0.0,60,1) AS near ON
g.ObjID=near.ObjID
WHERE ( (qualb > 0 AND qualb & 16 = 16) OR (qualr1 > 0 AND qualr1 & 16 = 16)
OR (qualr2 > 0 AND qualr2 & 16 = 16) OR (quali > 0 AND quali & 16 = 16) )
ORDER BY ra,dec
```

[\[Link to demo result set\]](#)

Q11: Find the plate numbers of those plates with nominal centres within 20 degrees of (185,0)

The nominal centres of all the survey fields are stored in the *Field* table, so this requires a join between that and the *Plate* table on the *fieldID* attribute. The spatial selection is performed by making a call to the table-valued function *fGetNearbyObjEq* in a subquery and then joining that *HTMID*, too.

```
SELECT TOP 30 platenum
FROM Plate AS p, Field AS f
WHERE p.fieldID=f.fieldID
AND f.HTMID IN
(SELECT HTMID FROM fGetNearbyObjEq(180.0,0.0,1200.0,5))
ORDER BY platenum
```

[\[Link to demo result set\]](#)

Q12: Find the positions and (B-R,R-I) colours of all galaxies with blue band area between 100 and 200 pixels, -20 < supergalactic latitude (sgb)/degrees < 20, and declination less than zero, and return them in colour order.

This repeats one of the SDSS “20 Queries”, but replacing a surface brightness cut with an image area cut, since surface brightness values are not stored within the SSA. The *Source* table - and, hence, the *ReliableGalaxies* view - includes positions in Galactic coordinates, so this query just requires a little spherical trigonometry to implement the selection on supergalactic latitude, and it requires a join with *Detection* to retrieve the B Band area. The clause containing the cut on supergalactic latitude is made more cumbersome by the requirement of converting angles between degrees and radians several times.

```
SELECT TOP 30 g.ra,g.dec, gCorMagB-gCorMagR2 AS BR, gcorMagR2-gCorMagI
AS RI
FROM ReliableGalaxies AS g, Detection AS d
WHERE d.objID=g.objIDB AND d.objID>0
AND d.area BETWEEN 100 AND 200
AND g.dec < 0
AND ABS(DEGREES(ASIN(SIN( RADIANS (6.32))*SIN (RADIANS (b))-COS
(RADIANS (6.32))*COS (RADIANS (b))*SIN (RADIANS (l)-137.37))))<20.0
ORDER BY BR,RI
```

[\[Link to demo result set\]](#)

Q13: Find the positions, R band magnitudes and B-R colours of all galaxies with an area greater than 100 pixels and a major axis 10 < d/arcsec < 30 in the red band and with an ellipticity>0.5.

The SuperCOSMOS plate scale is 67.14 arcsec/mm, so a conversion factor of 1 micron =0.067 arcsec is required to recast this query in terms of the physical units in which the length of the major axis of an image is recorded in the *Detection* table. The main constraints in this query are applied to rows in the *Detection* table and the join with the *ReliableGalaxies* table is made only to remove duplicate entries in the result set, as a consequence of the two epochs of red survey data, and to extract the colour information. The constraint that *sourceID* >0 is required to remove the parents

of deblended images, as these are stored in *Detection* for completeness, but only their children are merged to form entries in *Source*.

```
SELECT TOP 30 g.ra AS RA,g.dec AS Dec,
gCorMagR1 AS R1, gCorMagR2 AS R2, gCorMagB-gCorMagR1 AS
BR1,gCorMagB-gCorMagR2 AS BR2
FROM Detection AS d, ReliableGalaxies AS g
WHERE (area > 100) AND (au*0.067 BETWEEN 10 AND 30) AND (1.0-bu/au) >
0.5
AND ((g.objidr1=d.objid) or (g.objidr2=d.objid)) AND d.sourceID > 0
ORDER BY g.ra,g.dec
```

[\[Link to demo result set\]](#)

Q14: Find galaxies that are blended with a star and output the deblended magnitudes.

Entries in the *Detection* table which were blended together can be recognised by having a common parent, so this query looks for such pairs of objects by performing a self-join on *Detection* constrained to pick galaxies from one copy of the table and stars from the other, with added checks that they have sensible magnitudes and that duplicate records in the overlap regions between plates are excised.

```
SELECT TOP 30 g.objID,g.gMag,s.sMag
FROM Detection AS g, Detection AS s
WHERE g.class=1 AND s.class=2
AND g.parentid = s.parentid
AND g.parentID != 0
AND g.gMag > 0 AND s.sMag > 0
AND ((s.seam = 0 and g.seam=0) OR (s.seam=s.ssafield AND g.seam=g.ssafield))
ORDER BY g.objID
```

[\[Link to demo result set\]](#)

Q15: Find all pairs of galaxies within 10 arcsec of another that have very similar colours, and return their positions and B band magnitudes

The original SDSS query on which this is based is motivated by a gravitational lens search. For the SSA, we take “similar colours” to mean B-R and R-I colours within 0.1 mag, and execute it solely over the magnitude range $17.0 < B_J < 18.0$,

where the star/galaxy separation and photometric accuracy are both very good. This query uses the SSA *Neighbours* table, which records which entries in *Source* are less than 10 arcsec from which other ones, and which is designed to speed up queries like this, which would otherwise require very time-consuming self-joins of *Source* with a spatial match predicate. Note that the find clause prevents double-counting.

```
SELECT TOP 30 g1.ra AS RA1, g1.dec AS Dec1, g2.ra AS RA2, g2.dec AS Dec2,
g1.classMagB AS B1, g2.classMagB AS B2
FROM ReliableGalaxies AS g1, ReliableGalaxies AS g2, Neighbours AS n
WHERE
((g1.classMagB BETWEEN 17.0 AND 18.0) AND (g1.classMagR2>0.0) AND
(g1.classMagI>0.0)) AND
((g2.classMagB BETWEEN 17.0 AND 18.0) AND (g2.classMagR2>0.0) AND
(g2.classMagI>0.0)) AND
ABS ((g1.classMagB-g1.classMagR2)-(g2.classMagB-g2.classMagR2)) < 0.1 AND
ABS ((g1.classMagR2-g1.classMagI)-(g2.classMagR2-g2.classMagI)) < 0.1 AND
(g1.objID=n.objID) AND (g2.objID=n.neighbourobjID) AND
g1.objID > g2.objID
ORDER BY g1.ra,g1.dec
```

[\[Link to demo result set\]](#)

Q16: Find the positions of stars with Sloan 5-band colours and SSA proper motions which are consistent with their being subdwarfs.

This query roughly mirrors the work of [Digby et al \(2003\)](#), who constructed a sample of subdwarf stars on the basis of their location in a reduced proper motion (RPM) diagram computed using SSA proper motions, μ , together with r band magnitudes and $(r-i)$ colours from the SDSS Early Database Release (EDR) catalogue. The reduced proper motion, H_r , is defined to be $H_r = r + 5 \cdot \log_{10}(\mu/\text{arcsec per year}) + 5$ and then the Digby et al (2003) selection criteria for subdwarfs roughly translates to that region of the RPM diagram bounded by the lines $H_r > 13.3 + 6.5 \cdot (r-i)$ and $H_r < 15.3 + 6.35 \cdot (r-i)$. The query is a three-way join between the *ReliableStars* view of the SSA, the *PhotoObj* table of SkyServer and the *CrossNeighboursEDR* table, which records, for each entry in *Source*, its *ObjID* value, together with the *ObjID* value in *PhotoObj* of each SDSS object lying within 10 arcsec of it, as well as their separation (in arcmin): *CrossNeighboursEDR* therefore facilitates spatial matching between *Source* (and, hence, *ReliableStars*) and *PhotoObj*, by avoiding the need to perform an on-the-fly join between the two full tables whenever a match is required. This query matches those stars from the SSA with good proper motions with SDSS stars lying within 1 arcsec, and then uses SDSS magnitudes and SSA proper motions to determine which lie within the region of the RPM diagram populated by subdwarfs.

```
SELECT TOP 30 sdss.ra, sdss.dec
FROM ReliableStars AS ssa, skyserverv3.dbo.photoobj AS sdss,
CrossNeighboursEDR as crossn
WHERE
crossn.sdsstype = 6 AND sdss.mode=1
AND ((SQUARE (muacsd) + SQUARE (mud)) > 5.0 * SQRT ( SQUARE
(muacsd*sigmuacsd) + SQUARE (mud*sigmud) ))
AND sdss.psfmag_r + 5 * LOG10 (SQRT ((SQUARE (ssa.muacsd/1000.0))+
SQUARE (ssa.mud/1000.0))) + 5 > 13.3 + 6.5 * (sdss.psfmag_r - sdss.psfmag_i)
AND sdss.psfmag_r + 5 * LOG10 (SQRT ((SQUARE (ssa.muacsd/1000.0))+
SQUARE (ssa.mud/1000.0))) + 5 < 15.3 + 6.35 * (sdss.psfmag_r - sdss.psfmag_i)
AND sdss.objid=crossn.sdssid AND ssa.objid=crossn.ssaid
AND crossn.distancemins < 1.0/60.0
ORDER BY sdss.ra, sdss.dec
```

[\[Link to demo result set\]](#)

Q17: Provide a list of positions of galaxies whose Sloan and SSA magnitudes are consistent with there having been a supernova in the galaxy at one of its epochs of observation.

This query was motivated by the idea of using the epoch difference between Sloan and SuperCOSMOS surveys to estimate the local supernova rate, through identifying galaxy images which appear to have changed in brightness by more than five magnitudes; of course, in practice, this identifies more artefacts in the data than it does real supernovae, but several have been found this way. As with Q3, and subsequent queries, this requires use of colour equations to transform between the SDSS and SSA photometric system. This is another query that uses the *CrossNeighboursEDR* table to match (good quality) SSA galaxies with SDSS galaxies lying within 1 arcsec, and then further constraints on colour and magnitude are applied to find those which have changed in blue or red band brightness by more than 5 mags.

```
SELECT TOP 30 ssa.ra,ssa.dec
FROM ReliableGalaxies as ssa, skyserverv3.dbo.photoobj as sdss,
CrossNeighboursEDR as crossn
```

```

WHERE
(sdss.type = 3)
AND (crossn.ssa_id = ssa.objid) AND (crossn.sdssid = sdss.objid)
AND (crossn.distancemins < 1.0/60.0)
AND
(
ABS (sdss.modelmag_g - (ssa.classMagB-0.24*(ssa.classMagB-
ssa.classMagR2)+0.36))>5)
OR
ABS (sdss.modelmag_r - (ssa.classMagR2+0.08*(ssa.classMagB-
ssa.classMagR2)+0.13))>5)
)
ORDER BY ssa.ra,ssa.dec

```

[\[Link to demo result set\]](#)

Q18: Provide a count of high-quality star-like sources brighter than 16th magnitude which are in either the SSA but not in the SDSS.

This speculative query looks for bright optical transients, but, again, is more likely to turn up artefacts in the data, such as saturated bright stars. (*N.B.* we run this on the Personal SSA and the full EDR, and must be careful which way round we use the two databases, so that we don't just detect objects in the EDR lying outside its overlap with the PSSA.)

```

SELECT COUNT (*)
FROM ReliableStars
WHERE
((sCorMagB BETWEEN 0 AND 16) OR (sCorMagR1 BETWEEN 0 AND 16) OR
(sCorMagR2 BETWEEN 0 AND 16) OR (sCorMagB BETWEEN 0 AND 16))
AND objid NOT IN (SELECT ssalD FROM CrossNeighboursEDR WHERE
distancemins<0.5/60.0)

```

[\[Link to demo result set\]](#)

Q19: Provide the positions of star-like objects with SDSS colours consistent with being a quasar and positions consistent with not having moved between all the epochs in the SSA.

This query is motivated by the idea of selecting quasars as stellar-like objects with zero proper motion. The astrometric accuracy of SuperCOSMOS and the length of the time baseline between the SSA's constituent surveys are *nearly* adequate for this to be a sensible way to select quasar candidates, but it is mainly included as another example of an SSA-SDSS join query, using *CrossNeighboursEDR* and a colour-magnitude cut in the Sloan bands suitable for low-*z* quasars, namely $g \leq 22$, $-0.27 \leq u-g < 0.71$, $-0.24 \leq g-r < 0.35$, $-0.27 \leq r-i < 0.57$ and $-0.35 \leq i-z < 0.70$.

```

SELECT TOP 30 ssa.ra,ssa.dec
FROM ReliableStars AS ssa, skyserverv3.dbo.photoobj AS sdss,
crossneighboursEDR AS crossn
WHERE
(sdss.modelmag_g <= 22)
AND ((sdss.modelmag_u > 0) AND (sdss.modelmag_g > 0) AND (sdss.modelmag_r
> 0) AND (sdss.modelmag_i > 0) AND (sdss.modelmag_z > 0))
AND ((sdss.modelmag_u-sdss.modelmag_g) BETWEEN -0.27 AND 0.71)
AND ((sdss.modelmag_g-sdss.modelmag_r)BETWEEN -0.24 AND 0.35)
AND ((sdss.modelmag_r-sdss.modelmag_i) BETWEEN -0.27 AND 0.57)
AND ((sdss.modelmag_i-sdss.modelmag_z) BETWEEN -0.35 AND 0.70)
AND (ssa.sigmuacsd > ABS (ssa.muacsd))
AND (ssa.sigmud > ABS (ssa.mud))
AND crossn.ssa_id=ssa.objid AND crossn.sdssid=sdss.objid AND

```

```
crossn.distancemins < 0.5/60.0
```

```
ORDER BY ssa.ra,ssa.dec
```

[\[Link to demo result set\]](#)

Q20: Provide a list of SSA objects within a magnitude of their respective nominal plate limit which are unpaired in the SSA and have no SDSS counterpart.

This query is designed to detect spurious objects in the SSA, on the assumption that all real SSA objects should be detected in the (deeper) SDSS. It works by selecting SSA objects in *Source* which are detected in only one band, and within a magnitude of the respective nominal plate limit for that band, and then looks up the *ObjID* values for those sources to see whether they are included in the list of *ObjID* values for *Source* entries which have no SDSS counterpart with 0.5 arcsec, as revealed by the *CrossNeighboursEDR* table.

```
SELECT TOP 30 ra,dec
```

```
FROM Source
```

```
WHERE
```

```
(
```

```
((classMagB BETWEEN 21.5 AND 22.5) AND objIDR1=0 AND objIDR2=0 AND  
objIDI=0)
```

```
or ((classMagR1 BETWEEN 19.5 AND 20.5) AND objIDB=0 AND objIDR2=0 AND  
objIDI=0)
```

```
or ((classMagR2 BETWEEN 20.0 AND 21.0) AND objIDB=0 AND objIDR1=0 AND  
objIDI=0)
```

```
or ((classMagI BETWEEN 18.5 AND 19.5) AND objIDB=0 AND objIDR1=0 AND  
objIDR2=0)
```

```
)
```

```
AND objid NOT IN
```

```
(SELECT ssaid FROM CrossNeighboursEDR WHERE distancemins < 0.5/60.0)
```

```
ORDER BY ra,dec
```

[\[Link to demo result set\]](#)

5. The field 287 database

NB: if you are unfamiliar with the SSA and/or SQL, we recommend you read Sections 1, 2 and 3 of this manual before continuing. Furthermore, if you are not confident about being able to use SQL to get what you want out of the database, try the [analysis tool client GUI](#) which has push-button functionality to explore the dataset and also allows you to see the underlying SQL being generated to query the database as it produces the tables and plots.

The field 287 database contains a special time-resolved dataset constructed from the detections on nearly 200 plates in the same survey field, and sits alongside the main SSA database. It shares many of the same features, including a similar relational design, but is geared towards generation of accurate light-curve information for the restricted set of sources in the 6x6 square degree field. The time baseline covered is from ~1977 to ~2002.

5.1 Relational organisation

The database name for the field 287 dataset is "F287". You can access the database through the standard SSA free-form SQL web form by prefixing all object names with the string *F287..* (note there are **two** dots), e.g. to get the first 10 detections and their associated plate metadata,

```
SELECT TOP 10 d.*, p.* FROM F287..Detection AS d, F287..Plate AS p WHERE  
p.plateNum = d.plateNum
```

(Note that it doesn't matter which database - SSA or Personal SSA - you select in the drop-down box on the freeform SQL webform if you prefix database object names in this way).

In common with the SSA, F287 contains **Detection**, **Source** and **Plate** but there is no field table (because the plates all come from one field) and the merged sources are created from stack detections only. There are four stacks created from the best 8 plates from the middle of the run of individual plates in each of the passbands U, B, R and I. Unique identifiers relate each detection to its plate (*plateNum* in both *Plate* and *Detection* tables) and each source (*Source.objIDU*, *objIDB*, *objIDR* and *objIDI*) to its constituent stack detections (*Detection.objID*). Note that while *plateNum* is the standard UK Schmidt plate identifier without the passband letter for individual plates (i.e. 12602 for plate J12602 etc.), "plateNum" for the stacks is 1, 2, 3 and 4 for U, B, R and I respectively (this assignment being consistent with filterIDs assigned for those passbands). To see some details of the stacks,

```
SELECT plateNum, instrument, softVersion, filterID, raPnt, decPnt, objNum
FROM F287..Plate
WHERE plateNum IN (1,2,3,4)
```

The idea behind making a stack from a subset of all available plates in each passband is to create a "master" source list that is deeper than the deepest single plate in any passband; to crossmatch this list (see below) to the *Detection* table to link individual epoch measurements to their source; and to provide a reference for the relative recalibration of the photometry in the individual epochs to remove any systematic errors in that photometry as a function of field position and magnitude (such errors are large and omnipresent on Schmidt photographic plates owing to emulsion sensitivity variations). For example, to pick out one source and its corresponding stack detections, we use the following SQL:

```
SELECT s.*, u.*, b.*, r.*, i.*
FROM F287..Source AS s,
      F287..Detection AS u, F287..Detection AS b, F287..Detection AS r,
      F287..Detection AS i
WHERE s.objIDU = u.objID AND s.objIDB = b.objID AND s.objIDR = r.objID AND
      s.objIDI = i.objID
      AND s.objID = 3972
```

for example for source unique identifier (objID) number 3972. Note that the results summary page of the web interface will allow production of thumbnail images from the main SSA survey plates (which are included in the F287 set) if the results set includes non-default RA and Dec attributes.

5.2 Source environment from the neighbour table

The seeing in the F287 plate collection is not the best (typically 2 to 3 arcsec) in comparison to modern digital surveys like SDSS, and source confusion can be a problem. For example, close pairs of objects may be blended in the deep stack and some of the poorer individual epoch plates, and yet resolved and/or deblended on better individual plates. This means that the positional association of stack sources with epoch detections can be subject to significant numbers of mismatches. The same blending and quality information (except halo/spike/track flagging) as exists in the SSA is available to filter out deblends at query time; additionally, we provide a source neighbour table *SourceNeighbours* that can be used, amongst other things, to examine the environment of a source to see if there any close pairs. For example:

```
SELECT TOP 30 masterObjID, COUNT(*)
FROM F287..SourceNeighbours
GROUP BY masterObjID
```

shows the first thirty sources that have one or more neighbour(s) within the default neighbourhood radius of 10 arcseconds (this could be reduced to say 5 arcseconds by adding *WHERE distanceMins < 5.0/60.0*). Such a query could be used as a subquery filter (without the *TOP 30* and the aggregate function, of course) to remove those sources having crowded environs, e.g.:

```
SELECT COUNT(*)
FROM F287..Source AS s LEFT OUTER JOIN (
    SELECT masterObjID FROM F287..SourceNeighbours GROUP BY
    masterObjID
) AS t ON s.objID = t.masterObjID
WHERE t.masterObjID IS NULL
```

returns a count of all sources that have no neighbour within 10 arcsec.

5.3 Cross-neighbours

The association between *Source* and individual epoch detections in *Detection* is done through the cross-neighbour table *SourceXDetection*. Every source will have one or more detections recorded in *SourceXDetection* along with the angular displacement between the two records in arcminutes (*distanceMins*). This is because, at the very least, an isolated faint source detected in only one passband stack but in no other stacks (or the less deep individual epoch plates) will have a single crossneighbour record pointing to that individual stack detection, and a *distanceMins* of zero. The cross-neighbour association is always between the unique identifiers (also the relational primary keys) of the two joined tables for good query-time performance. In addition to the cross-neighbour table between *Source* and *Detection*, the F287 database also contains a cross-neighbour table between *Source* and a locally held copy of the Veron catalogue of known quasi-stellar objects.

The construction of the cross-neighbour tables is simple: for a given source in the master list *Source*, every object on each slave plate (including the stacks) within the neighbourhood radius of 10 arcseconds is located and recorded in the table via a row consisting of the master *Source* objID, the slave *Detection* objID, and the angular distance between them. It is via the *SourceXDetection* cross-neighbour table that light curves can be generated, while F287 source identifications of known QSOs can be found via the *SourceXKnownQSO* table - see below for further details and examples. The following SQL illustrates how to use the known QSO cross-neighbour table to find the F287 source IDs and names of a set of quasars having redshift 4 or greater:

```
SELECT masterObjID AS objID, q.*
FROM F287..KnownQSO AS q, F287..SourceXKnownQSO AS x
WHERE q.qsoID = x.slaveObjID AND z > 4 AND distanceMins < 3.0/60.0
```

Note that we specify that the crossmatch distance must be less than 3 arcsec, which is a good rule-of-thumb for data of this type. Another join via addition of *F287..Source* AS *s* in the FROM clause along with ... *AND s.objID = x.masterObjID* in the WHERE clause allows the user to see the F287 source information by also specifying ..., *s.** in the SELECT list.

5.4 Selection of variability candidates

Another major difference between the source tables of the SSA and F287 DBs is that the latter contains statistical summary attributes of all available epoch photometry for that particular source. Using a neighbourhood radius of 1 arcsecond and the *SourceXDetection* table, calculations of attributes like mean, standard deviation, median-absolute-deviation (MAD) and skewness have been made in the construction of the database so that the user can select samples of objects with certain variability characteristics (or indeed, samples of objects exhibiting no

variations in photometry within the likely errors). For example, typical photometric errors for point sources in the B band in a specific magnitude range can be extracted using *Source* attribute *madScorMagB* - fire the results of

```
SELECT 1.48*madScorMagB AS robustRMS
FROM F287..Source
WHERE sCorMagB BETWEEN 15.0 AND 16.0 AND madScorMagB > 0 AND
meanClass=2
```

into **TOPCAT** and plot a histogram (if you select either FITS or VOTable output formats on the SQL webform, the results page will contain a push-button link to start this handy application automatically). It is clear that the typical (modal) error at these magnitudes is ~ 0.05 mag. (Note the use of the scaling of MAD via a factor 1.48 to an equivalent Gaussian RMS assuming a normal distribution). We could then trawl for a sample of variable candidates with individual standard deviations (as a reasonably sensitive detector of even a few non-Gaussian outliers amongst the run of photometric measurements, in this case limited to at least 50) being 6 times larger than this, e.g.

```
SELECT *
FROM F287..Source
WHERE sCorMagB BETWEEN 15.0 AND 16.0 AND sigsCorMagB > 0.3 AND
meanClass = 2 AND numObsB >= 50
ORDER BY sigsCorMagB DESC
```

By this simple analysis, the most variable source is objID = 741793. However, the user should beware that more often than not, "variables" selected in this way may not be genuine - there are all manner of reasons why the run of photometric measurements should contain one or more outlying points with respect to the nominal errors - it is a fact of life that spurious measurements from emulsion flaws, deblending errors, and many other effects can lead to a non-Gaussian tail in the distribution of measurements. It is most advisable to use the various morphological attributes to filter out as many false positives as possible. For example, we could insist that the candidates are not deblended and have good **quality** in all stacks by adding ... *AND qualU < 64 AND qualB < 64 AND qualR < 64 AND qualI < 64 AND blendU = 0 AND blendB = 0 AND blendR = 0 AND blendI = 0* to the *WHERE* clause in the last query above. Then objID 690728 is the most variable candidate; note that while *sigsCorMagB* is large, *madScorMagB* is only ~ 2 sigma outlying. The latter is a robust Gaussian RMS equivalent sigma via scaled median of absolute deviation, and is less susceptible to the odd outlying point in the sequence. So the likelihood is that the light curve of this object will show just a few outlying points.

5.5 Light curves

The B band light curve data for the example in the previous section can be extracted as follows:

```
SELECT epoch, d.sCorMag AS B
FROM F287..SourceXDetection AS x, F287..Detection AS d, F287..Plate AS p
WHERE x.masterObjID = 690728 AND x.slaveObjID = d.objID
      AND d.filterID = 2 AND d.plateNum = p.plateNum AND p.plateNum > 4 AND
distanceMins = (
      SELECT MIN(distanceMins) FROM F287..SourceXDetection WHERE
masterObjID = x.masterObjID
      AND distanceMins <= 1.0/60.0 AND (slaveObjID/0x100000000) =
p.plateNum)
ORDER BY epoch ASC
```

where the subquery construct ensures that only the nearest neighbour within 1 arcsecond on each plate is considered, and the final clause orders the data by

ASCending time. (Note that inside the subquery we employ a feature of the construction of the *Detection* objIDs: these are assigned by running number in the four least significant bytes of the 8 byte integer, and with the plate number in the four most significant bytes. So dividing by the hexadecimal constant 0x100000000 right-shifts the bits by four bytes and leaves the plate number as the result. Although somewhat esoteric, this feature enables you to access the plate number of any objID without an SQL join with the *Detection* table.)

Finally, the table of known QSOs stored and crossmatched in the F287 database is provided to allow the study of the variability of the quasars in the field. For example, to find the objIDs of QSOs unambiguously identified by proximity within 1 arcsecond and having redshift more than 3.0, use the following query:

```
SELECT name, masterObjID AS objID, z, s.ra, s.dec
FROM F287..Source AS s, F287..SourceXKnownQSO AS x, F287..KnownQSO AS q
WHERE s.objID = x.masterObjID AND x.slaveObjID = q.qsoID AND distanceMins
<= 1.0/60.0 AND z > 3
```

It is then a simple matter to get the light curve in any colour by using the first query in this section, e.g. for object H 87 substitute s.objID = 154058. By combining elements from several of the above SQL fragments, we could get the R light curve of H 87 in one go via:

```
SELECT epoch, d.sCorMag AS R
FROM F287..KnownQSO AS q, F287..SourceXKnownQSO AS qx,
F287..SourceXDetection AS x, F287..Detection AS d, F287..Plate AS p
WHERE name = 'H 87' AND qx.masterObjID = x.masterObjID AND x.slaveObjID =
d.objID AND q.qsoID = qx.slaveObjID
AND d.filterID = 3 AND d.plateNum = p.plateNum AND p.plateNum > 4 AND
qx.distanceMins <= 1.0/60.0 AND x.distanceMins = (
SELECT MIN(distanceMins) FROM F287..SourceXDetection WHERE
masterObjID = x.masterObjID
AND distanceMins <= 1.0/60.0 AND (slaveObjID/0x100000000) =
p.plateNum)
ORDER BY epoch ASC
```

6 The 2MASS Photometric Redshift Catalogue

The 2MASS photometric redshift catalogue is not an archive of data from the UKST and POSS photographic plates measured by SuperCOSMOS, but is a value added product of calculated photometric redshifts derived from 2MASS and WISE as well as the SuperCOSMOS data. The above queries are not relevant, but this table is very useful in conjunction with wide area surveys such as UKIDSS-LAS, VISTA-VHS.

6.1 Matching 2MPZ to other surveys

2MPZ can be matched to the 3 input catalogues, TWOMASS..twomass_xsc, SSA..Source and WISE..wise_allskysc via the integer IDs, e.g. to get the julian date and hemisphere of the twomass observation:

```
SELECT pz.twomassID, pz.twomassX, pz.zPhoto, x.jdate, x.hemis
FROM twompzPhotoz AS pz, TWOMASS..twomass_xsc AS x
WHERE x.ext_key = pz.twomassID
```

2MPZ can be matched to older versions of the UKIDSS and VISTA surveys through the 2MASS XSC neighbour table, e.g. UKIDSS_LAS DR10, via [the WFCAM Science Archive](#):

```
SELECT s.sourceID,s.mergedClass,s.kPetroMag,s.kPetroMagErr,pz.zPhoto
FROM lasSource AS s, lasSourceXtwomass_xsc as x, TWOMPZ..twompzPhotoz
as pz
WHERE s.sourceID=x.masterObjID and x.slaveObjID = pz.twomassID and
x.distanceMins<0.033 and x.distanceMins in (select min(distanceMins) from
lasSourceXtwomass_xsc where masterObjID=x.masterObjID)
```

[Home](#) | [Overview](#) | [Browser](#) | [Access](#) | [Cookbook](#) | [Links](#) | [Credits](#)
[Radial](#) | [MenuQuery](#) | [FreeSQL](#) | [CrossID](#)

WFAU, Institute for Astronomy,
Royal Observatory, Blackford Hill
Edinburgh, EH9 3HJ, UK
Tel +44 131 668 8356 (office)
or +44 131 668 8100 (switchboard)

mar@roe.ac.uk
6/11/2013